Auditing Actual Execution and Output Portability in Quantized LLM Inference: A Measurement Study of llama.cpp

Shashvath Bhaskar
Product Lead, Oxmiq Labs
shashvath.bhaskar@oxmiq.ai

October 2026 (technical note)

Abstract

I measured whether the same quantized model, the same prompt, and greedy decoding produce the same output across the compute backends of llama.cpp. The study pins one release (b11327, official binaries, SHA256-recorded) and runs it on 12 machines spanning NVIDIA, AMD, and Intel GPUs on both Linux and Windows. The decisive methodological step turned out to be auditing *which device actually executed each run*. I screened every (machine, backend) configuration with per-prompt wall-time ratios against the same host's CPU build, and corroborated a subset directly on the box with process maps, VRAM readings, and library-load checks. The audit found three failure classes in which the server silently ran on the CPU while presenting as a GPU backend. No GPU-related console output appeared in any fallback run I diagnosed. The first class affects the official ROCm build on all four AMD hosts: its backend library needs ROCm userspace SONAMEs one major version newer than the stock ROCm 6.3.0 installation on the host I diagnosed directly, and the release bundles no Linux ROCm runtime. The second was my own harness mistake, a missing runtime-library path. The third was a virtualized host with no access to its advertised NVIDIA GPU. On runs the audit classified as GPU-executed, byte-exact agreement with the same-host CPU reference rises with precision: F16 88.0%, Q8_0 35.3%, Q4_K_M 18.0% over 8,670 prompt comparisons (34 raw-completion prompts per cell, 128-token cap). The same precision ordering appears between the GPU builds' shared CPU implementation and the standalone CPU build (F16 92.7%, Q8_0 40.5%, Q4_K_M 17.7%), and between hosts from different CPU equivalence classes. Divergence concentrates at near-ties: among 7,875 first divergences, the alternative token lies inside the reference's recorded top-5 in 99.89% of events (median logprob gap 0.062 nats). Both stability controls reproduced every generation they tested: 578/578 comparisons across two identical-stack machines, and 1,734/1,734 in a same-machine 12-hour re-run. I release the harness, the per-token logprob records, and the audit method. The practical conclusions: verify what actually executed before trusting any backend label; do not assume exact-output portability for quantized models across numerical stacks at Q4-class precision; and do not use default-tool perplexity as a conformance signal.

1 Introduction

A greedy-decoded output is usually treated as a property of the model and the prompt. This note studies the third variable: the numerical stack that executes the model. I ask two questions. First, *does the intended backend actually execute the workload*? Second, *when the stack differs, how much does greedy output differ, and where*?

The first question became the pivot of the study, and not in the way I expected. I began with a straightforward backend comparison on a multi-vendor fleet, and I initially read the results as backend behavior. Then a device-truth audit — comparing each run's per-prompt generation wall time against the same host's CPU build — showed that three of my "GPU backend" columns had been serving from the CPU the entire time, without a fatal error. The official ROCm build had fallen back on all four AMD hosts. The CUDA-13 build had fallen back on both hosts offering it. CUDA had fallen back on one virtualized host.

The three failures have different owners. The first involves the upstream binary's runtime requirements. The second was a launch-environment mistake in my own harness, which passed the runtime-library path to device enumeration but not to the server process. The third was a virtual machine with no access to its advertised NVIDIA GPU. I treat them differently in what follows, and two of the three are my side of the ledger.

All three shared one visible behavior: the server kept serving HTTP requests normally. In every run I diagnosed directly, the console carried no GPU-related output at all — no error, no warning, no offload summary. None can have appeared in the sweep runs either, because their servers were never passed `-ngl`, the warning's only trigger. The silence is structural. Release builds suppress backend-load failures entirely. The engine's "no usable GPU found" warning is printed only when `-ngl` is passed explicitly, and even then it is suppressed whenever the bundled RPC backend is registered — as it was in every run I diagnosed. Device enumeration, meanwhile, does not by itself establish execution: ROCm enumeration returned no device, and in the CUDA-13 case enumeration succeeded under a different environment from the server.

The audit changed the study's claims in both directions. It led me to withdraw two findings I had briefly believed: a "CUDA-toolkit changes everything" comparison, and a "perplexity defaults differ between tools" hypothesis. The compared processes had not run under equivalent runtime environments or execution placements, and several of the comparisons had in fact measured the GPU builds' shared CPU implementation against the standalone CPU build. The audit also sharpened what survives: the agreement structure of runs it classified as differing in execution, the near-tie anatomy of divergence, and the complete agreement of the identical-stack controls.

2 Method

Figure 1 shows the measurement pipeline; this section describes each stage and why it is built that way.

Fleet. The fleet is 12 machines carrying 13 run labels: one machine was measured twice, 12 hours apart, and its re-run is used only as a stability control. One of the twelve is a virtualized guest. The NVIDIA side covers an RTX 5060 Ti, an RTX PRO 6000 Blackwell, a TITAN RTX, an RTX 3060 Ti on Windows, and the virtualized RTX 4090 guest that turned out to lack access to its advertised GPU (§3). The AMD side covers two RX 7900 XTX cards, a Radeon AI PRO R9700, and a Strix Halo 8060S integrated GPU. The Intel side covers an Arc A770 on Linux, an Arc A770 on Windows, and an Arc B60. The hosts span several CPU generations and both desktop operating systems, which is what makes the equivalence classes of §4 visible at all. I orchestrated the entire fleet from a single workstation through OxCapsule [13], a fleet-management platform for heterogeneous AI compute: each host ran one pinned, resumable sweep session, and records were checkpointed and archived after every machine.

Engine and models. Everything runs llama.cpp release b11327, official prebuilt binaries only, with every artifact's SHA256 recorded. Nothing is ever built locally, so every number in the study traces to a public, bit-identical binary. The model axis is 17 core GGUF files: Qwen2.5-Instruct

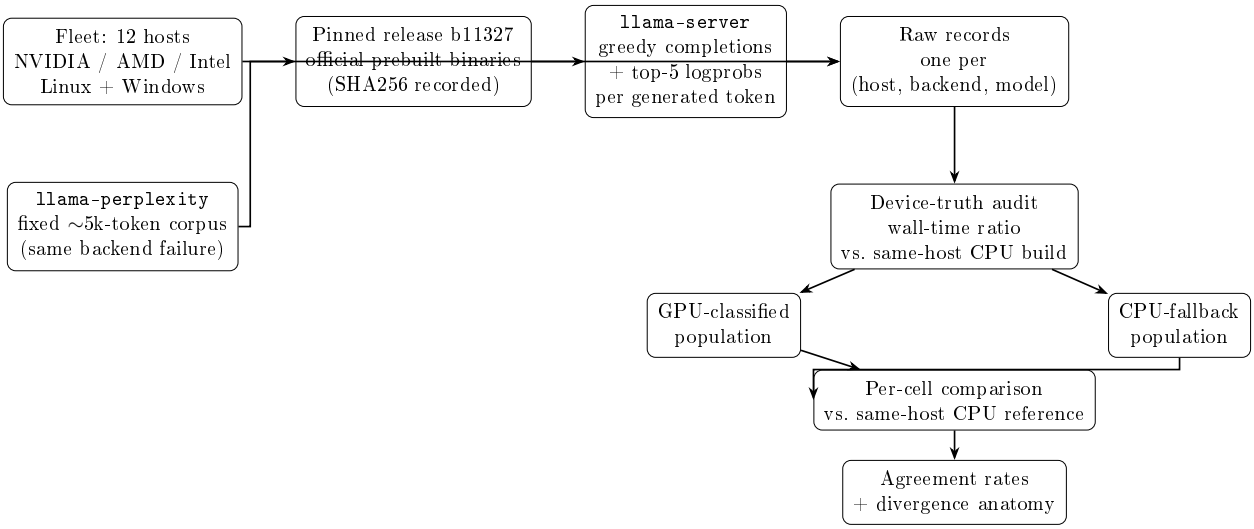


Figure 1: The measurement pipeline. Every host runs the same pinned release; the server protocol captures generations and per-token logprobs; the device-truth audit splits runs into GPU-classified and CPU-fallback populations before any comparison is made.

at 0.5B/1.5B/3B and SmolLM2-Instruct at 135M/360M/1.7B, each at Q4_K_M and Q8_0, plus F16/FP16 for the five models up to 1.7B (the 3B was not run at F16). The Qwen files are official Qwen-org quantizations; the SmolLM2 files are community re-quantizations, which is a caveat for family comparisons.

Protocol. Each cell — one (host, backend, model) combination — completes 34 fixed raw-completion prompts with greedy decoding (temperature 0), a 128-token cap, prompt caching disabled, and one request at a time. The prompts use no chat template, so every backend processes byte-identical input. The server protocol records, for every generated token on every backend, the chosen id, its logprob, and the top-5 alternative ids with logprobs; this is what makes the near-tie analysis of §6 possible. Each cell also measures perplexity with the same release's `llama-perplexity` on a fixed ∼5k-token corpus. The reference for all per-cell comparisons is the same host running the release's standalone CPU build: one binary on every Linux host, the Windows CPU build on the two Windows hosts. Using the same host's CPU as the reference keeps host effects out of the backend comparison — the cross-host structure of the CPU reference itself is analyzed separately in §4.

Device-truth audit. For every (machine, backend) configuration, I compute the median over models of the ratio of median per-prompt generation wall time to the same host's CPU build. The observed ratios split cleanly into two groups in this data: GPU-classified configurations sit at 0.06–0.66 and fallback configurations at 1.00–1.03, with nothing in between. Timing alone is screening evidence, so I corroborated the classification directly on-box for one configuration per failure class — process maps, VRAM readings, and library-load checks (§3) — plus one genuine-GPU CUDA configuration and three pinned Vulkan re-runs. The remaining classifications rest on the observed timing separation, corroborated by device listings in the later supplemental records (the core records predate that field). A configuration at ≈ 1.0 is classified *CPU-fallback* and analyzed as its own population, never as a GPU backend. Figure 2 shows the split for all 25 classified configurations.

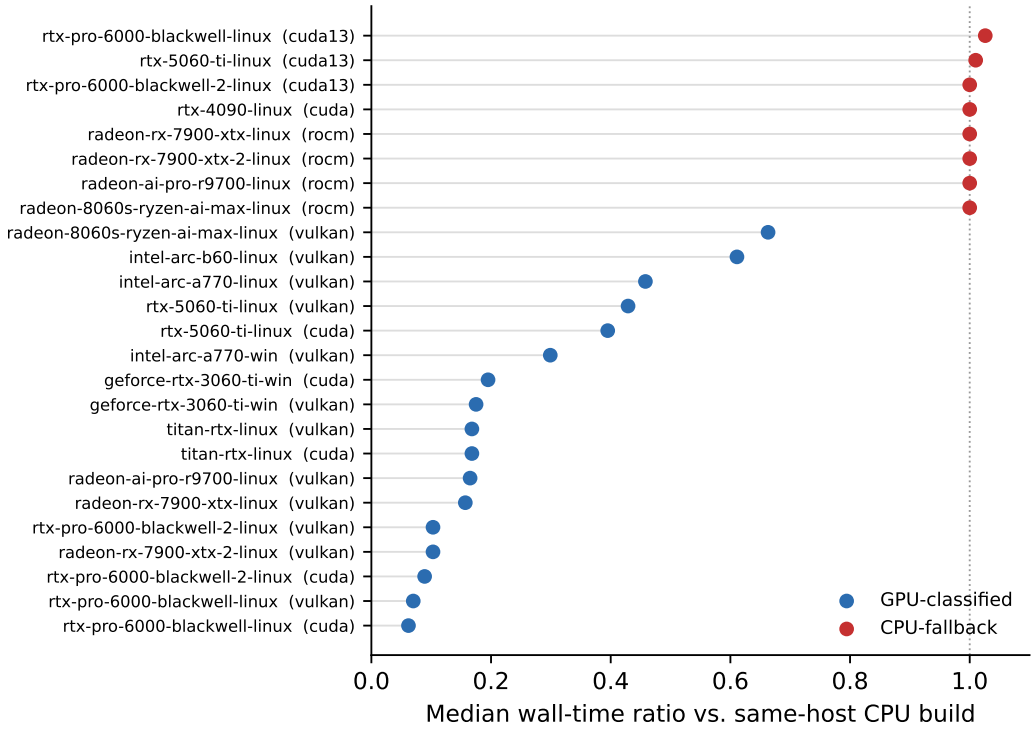


Figure 2: The device-truth audit's screening signal: median per-prompt wall-time ratio against the same host's CPU build, for every (machine, backend) configuration in the dataset. The split is clean — GPU-classified configurations (blue) sit at 0.06–0.66, fallback configurations (red) at 1.00–1.03, with nothing in between. Data: `results/audit-v2.json` (`devices`); figure regenerable with `harness/make_figs.py`.

3 Finding 1: silent CPU fallback

All three failure classes share one user-visible behavior. The server accepts requests and generates normally. In every fallback run I diagnosed, the console carried no GPU-related output at all — no error, no warning, no offload summary — not even when I passed an explicit `-ngl 999`. The engine's own source, at this release, explains why. Release builds load backends with failure messages suppressed. The "no usable GPU found" warning in `common/arg.cpp` fires only inside the `-ngl` option handler. And even there it is guarded by `llama_supports_gpu_offload()`, which returns true whenever the bundled RPC backend is registered — as it was in every run I diagnosed; its library appears in the process maps. The process serves on the CPU regardless.

(a) Missing backend userspace (upstream-relevant). The official ROCm release asset (named `rocm-10.0`) ships a `libggml-hip.so` that requires `libamdhip64.so.7`, `libhipblas.so.3`, and `librocblas.so.5`. The host I diagnosed directly, running the stock ROCm 6.3.0 userspace, provides only `.so.6`/`.so.2`/`.so.4` — one major version older. The SONAMEs do not match even with `/opt/rocm/lib` on the library path, and the release bundles no Linux ROCm runtime at all (a Windows runtime zip exists; the Linux equivalent does not). The backend library therefore never loads on the directly diagnosed host, an RX 7900 XTX. On the other three AMD hosts, the same failure follows from three independent signals: the CPU-class wall-time signature, perplexity equality at the tool's printed precision, and mutually bit-identical fallback outputs.

What does this do to a measurement? On all four hosts, `--list-devices` prints `(none)` (recorded per-record in the supplemental runs), and the server runs fully on the CPU — including with an explicit `-ngl 999`. Process maps show no HIP library, VRAM stays idle, and generation proceeds at approximately the standalone CPU build's speed. The perplexity tool suffers the same backend failure, so its values are host-CPU values: the "ROCm" perplexity matched the CPU build's in 68 of 68 cells, at the tool's printed precision. The "ROCm" *generations* did not match the standalone CPU build's byte-for-byte — 46.8% pooled agreement — because the fallback runs executed the GPU builds' bundled CPU implementation, which is shared across the ROCm, CUDA, and CUDA-13 release builds and differs from the standalone CPU build's. The fallback outputs are bit-identical across the three AMD hosts sharing a host-CPU class (578/578 pairwise), consistent with same-build, same-CPU-class execution.

Similar behavior has been publicly reported by other users for Windows and other setups on the project's public issue tracker. My contribution is the Linux SONAME detail, the cross-host confirmation, and the benchmarking consequence: quality measurements taken this way silently measure *a* CPU implementation — the fallback build's own, not the standalone CPU build's — while presenting as GPU measurements.

(b) Runtime-library environment mistake (mine). My harness passed the CUDA runtime-library path (`LD_LIBRARY_PATH`) to device enumeration and to the perplexity tool, but not to the server process. On hosts where `libcudart.so.13` exists only in the release's runtime tarball, the CUDA-13 backend therefore loaded for `--list-devices` — which reported the GPU — but not for the server, which served from the CPU without a fatal error. An on-box A/B check at matched settings (`-ngl 999` both times) isolates the single variable. With the environment set, the CUDA-13 build offloads at 602 tok/s with 456 MiB of VRAM in use. Without it, the same command runs on the CPU at 203 tok/s with 2 MiB in use. On the same host, the CUDA-12.8 build was unaffected because the system carries CUDA 12.9 libraries. I retract the preliminary "CUDA-13 behaves like a different backend" observation that this artifact produced. The general lesson: `--list-devices` output reflects the environment of whatever process ran it, which is not necessarily the environment of the server.

(c) Absent hardware. One host, labeled as a virtualized RTX 4090, has no NVIDIA device on its guest PCI bus at all: no `/dev/nvidia*`, no driver library, only a guest-visible Intel iGPU on `lspci`. Its CUDA runs are classified as CPU-executed by the audit. Its Vulkan runs are excluded from the study, because they enumerated and used the guest-visible Intel iGPU. I retain this host's CUDA cells only within the CPU-fallback population, and I note that my earlier "virtualized 4090" hardware label was simply wrong.

Detection. None of the three classes produced any GPU-related console output, even with an explicit `-ngl`: the registered RPC backend suppresses the engine's only GPU-availability warning, and release builds swallow backend-load failures silently. Enumeration is not the answer either. It correctly reported no device when the ROCm backend could not load — but in the CUDA-13 case it reported the GPU from a process running in a different environment than the server, so that result did not establish server offload. The wall-time-ratio audit caught all three classes, and it is cheap: it requires only a CPU-build baseline on the same host. I recommend it as a standing screening check in any benchmarking or conformance setup, followed by post-launch verification that the backend library is actually mapped in the server process.

4 Finding 2: conformance on runs classified as GPU-executed

On runs the audit classified as GPU-executed — CUDA on 4 machines (the 12.8 build on three Linux hosts, the 12.4 build on the Windows host) and Vulkan on 11, for 255 cells after removing the re-run — byte-exact agreement with the same-host CPU reference is:

Population	Q4_K_M	Q8_0	F16/FP16
GPU vs same-host CPU (n: 3060/3060/2550)	18.0%	35.3%	88.0%
GPU builds' shared CPU impl. vs standalone CPU (n: 1428/1428/1190)	17.7%	40.5%	92.7%
Same CPU build, same host-CPU class (22 host pairs)	100%	100%	100%
Same CPU build, different host-CPU class (23 pairs)	13.7–20.1%	35.3–39.2%	90.0–94.1%

Denominators are prompt comparisons. F16/FP16 covers five model files and Q4_K_M/Q8_0 cover six, because Qwen2.5-3B was not run at F16; cross-host rows compare 578 model–prompt combinations per pair (204 at Q4, 204 at Q8, 170 at F16). The second row's 4,046 comparisons collapse to 1,734 distinct (host-class, model, prompt) combinations — 1,269 distinct content pairs — because the GPU builds ship one shared CPU implementation, so fallback runs on hosts sharing a CPU class are bit-identical to each other.

Equivalence classes are empirical: a class is a group of hosts whose standalone CPU-build outputs are identical across all 578 combinations. Class A holds seven Linux hosts, including the TITAN RTX and PRO 6000. Class B holds the second 7900 XTX host and the 4090 guest. The 5060 Ti host is a singleton. The classes track CPU instruction-set tiers — class A hosts expose AVX-512 without AMX, class B hosts lack AVX-512, and the singleton carries AMX — and they are coarser than kernel-variant names: the two directly-mapped class-B hosts loaded different AVX2-tier CPU variants yet produced identical outputs. Cross-host rows use the Linux CPU binary only; the two Windows hosts agree with each other under the Windows CPU build and are excluded from these rows.

Three observations follow from the table (Figure 3 shows the precision structure visually).

Quantization dominates. Among the GPU/CPU, cross-build CPU, and cross-class CPU comparisons, agreement rises strongly with precision. The pooled "∼45–48% agreement" that a fixed quant mix produces is a mixture artifact: per-precision rates in these populations span roughly 14% to 94%. In contrast, every same-class CPU host pair agreed completely, at every precision.

The comparison type is a second-order effect. GPU-vs-CPU, the GPU builds' shared CPU implementation vs the standalone CPU build (including the ROCm and CUDA-13 fallback columns, reclassified), and cross-class CPU-vs-CPU all follow the same precision ordering. Excepting same-class host moves, byte-exact portability across the stack changes I tested is largely absent at Q4-class precision and partially preserved at F16.

Cross-host CPU comparisons are bimodal. Every pair is either bit-identical or diverges like any other stack change — nothing lands in between — and I group the identical pairs into the equivalence classes above (the Linux fleet has two nonsingleton classes and one singleton; the Windows pair also agrees internally). "Same result as CPU" therefore means "same result as this host's CPU build, and only hosts in its equivalence class".

5 Finding 3: agreement in identical-stack controls

Two stability controls bound the divergence findings. In the first, two physically distinct machines — same GPU (RX 7900 XTX), same OS, same kernel driver version (amdgpu 6.10.5; the Vulkan userspace version was not recorded), same Vulkan binary — produced bit-identical generations for

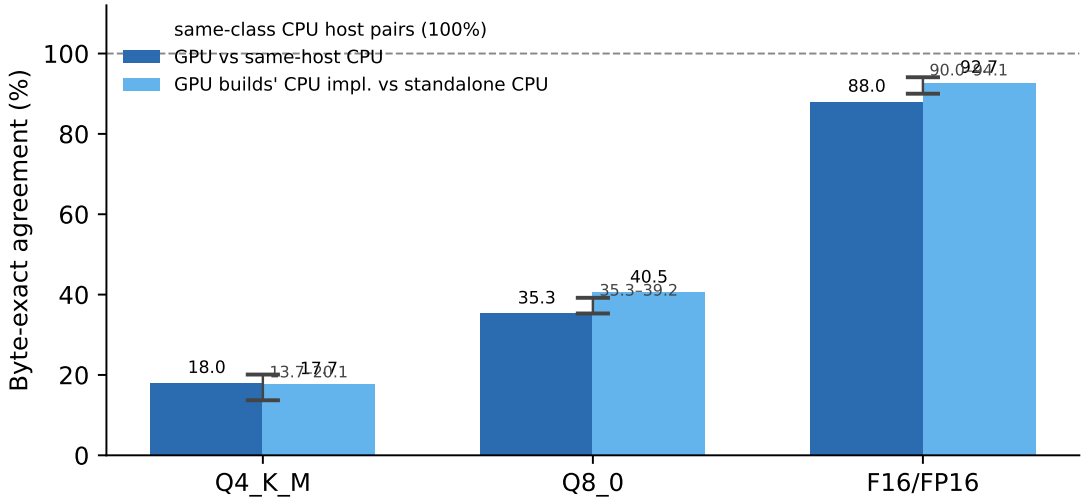


Figure 3: Byte-exact agreement by precision and population. Bars show the two primary populations; the short vertical lines span the cross-class CPU host-pair range; the dashed line marks the 100% agreement of every same-class CPU host pair. Data: `results/audit-v2.json` (`per_quant`); figure regenerable with `harness/make_figs.py`.

578/578 prompt comparisons. In the second, the same machine re-run 12 hours later (51 cells, 1,734 prompts across CUDA, CUDA-13, and Vulkan) reproduced its earlier outputs bit-for-bit in 100.0% of comparisons.

These controls demonstrate repeatability in the tested configurations, and they support reading the observed differences as stack-dependent. They do not establish determinism for any stack I did not test. (I note separately that the engine's own documentation warns that prompt-cache reuse can introduce run-to-run nondeterminism via batch-shape-dependent prefill/decode arithmetic[1]; my protocol runs with caching disabled.)

6 Finding 4: divergence anatomy

When two runs diverge, where do they diverge? The dataset answers at token level, because the server protocol recorded each side's top-5 logprobs for every generated position.

There are 7,875 first-divergence events in total (6,908 after removing the same-box re-run's 967 duplicate events; the figures below use the full set, and deduplication moves top-5 membership only from 99.89% to 99.90%). The events span the primary per-cell comparison populations: 4,796 GPU-vs-CPU, 2,112 build-CPU-vs-CPU, and 967 from the re-run machine's cells against its own CPU reference. In 99.89% of these events, the diverging backend's chosen token lies within the reference's recorded top-5. The exceptions are all in SmolLM2 Q4_K_M, at four (model, prompt) loci with 0-based prompt indices: the 9 reference-side exceptions sit at two loci (135M prompt 23; 360M prompt 26), the 9 backend-side exceptions at three (360M prompt 26; 135M prompt 22; 360M prompt 1), and the two sets overlap in 5 events.

How close are the competing tokens? Each directional gap is available for 7,866 events (both gaps jointly for 7,862 of them); the quantiles below use each direction's available events separately.

[1] `tools/server/README.md` in the b11327 release tree.

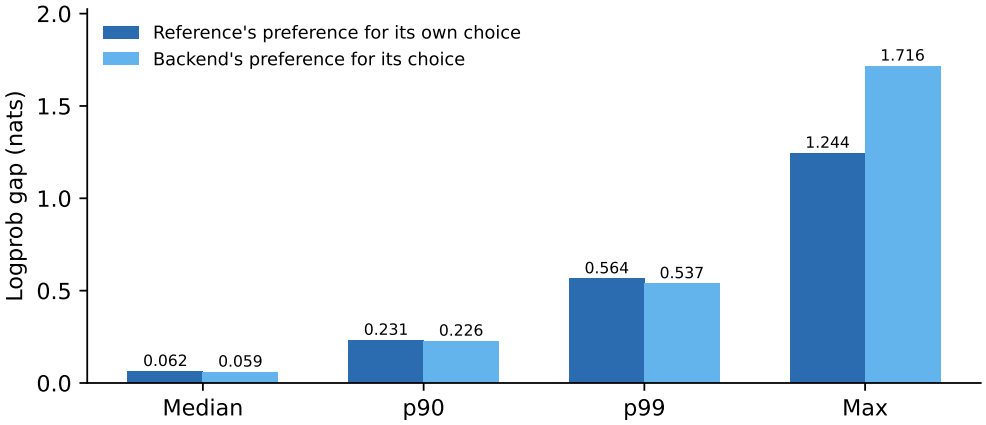


Figure 4: How close the competing tokens are at first divergence: quantiles of the logprob gap in both distributions (n = 7,866 events per direction). The median gap is about six hundredths of a nat — near-ties — and 90.0% of divergences are mutual rank-1↔2 swaps. Data: `results/audit-v2.json` (`gap_ref_quantiles`, `gap_backend_quantiles`); figure regenerable with `harness/make_figs.py`.

In the reference distribution, the preference for its own argmax over the backend's choice has median 0.062 nats (p90 0.231, p99 0.564). In the backend's distribution, the preference for its choice over the reference's has median 0.059 nats (p90 0.226). And in 7,090 of the 7,875 events (90.0%), the divergence is a mutual rank-1↔2 swap: each side's chosen token is the other's runner-up.

Divergence at greedy decoding is therefore concentrated at near-ties. This is compatible with the rounding-order mechanism assumed in prior work [7], though the data do not by themselves exclude small systematic implementation differences that would also preferentially flip low-margin decisions. Figure 4 summarizes the gap distributions.

7 Related work

Numerical deviation across platforms has a precedent outside LLMs: Schlögl et al. [1] established cross-platform numerical deviation in DNN inference for vision models, together with platform equivalence classes. He (Thinking Machines Lab, 2025) [2] identifies batch-shape dependence as a source of temperature-0 serving nondeterminism and ships batch-invariant kernels; the engine's own cache-reuse warning, cited above, is an instance of this class. Yuan et al. [3] measure greedy divergence across GPU type, count, batch size, and numerical precision, and propose LayerCast.

Closest to the measurements here, Du et al. [7] measure cross-precision greedy divergence with margin-thresholded mitigation, and Pape et al. [5] quantify backend-choice effects across engines including llama.cpp. Dutta et al. [4] show that aggregate accuracy metrics can hide flips in compressed models; my fallback observations add a distinct failure mode, in which a perplexity tool and a generation server run under different execution placements, so the aggregate metric is computed by a different implementation than the one being evaluated.

On deterministic or verified inference, Gond et al. [6] (LLM-42), Cooper et al. [8] (fixed-reduction GEMMs), and TOPLOC (Ong et al. [9]) address the problem from the systems side. In software engineering, CRADLE [10] cross-validated deep-learning library backends; my harness follows that tradition for autoregressive LLM inference and extends it with an execution-placement check. Training-side tooling variance is covered by Zhuang et al. [11] and Pham et al. [12].

8 Limitations

- **One engine, one release.** All results are for b11327. Rates may drift across releases; the fallback classes and the audit method are the durable content.
- **Small models.** Results cover models from 135M to 3B; portability to larger models is unmeasured.
- **No ROCm GPU execution.** No ROCm/HIP GPU run ever executed in this dataset — AMD GPUs appear in the GPU-classified population only via Vulkan.
- **Prompting.** Raw-completion prompts without chat templates may produce different output distributions from deployed chat prompting.
- **Mechanism.** The near-tie evidence is compatible with, but does not uniquely establish, rounding-order causation.
- **Perplexity.** Cross-backend perplexity comparisons are confounded by the fallback classes and by prefill-vs-decode kernel differences.
- **SYCL.** SYCL contributed no complete cells: the Linux binaries failed to load the oneAPI runtime, and the Windows build loaded, demonstrably offloaded, then crashed mid-run.

9 Conclusion

Four recommendations for practitioners follow from the measurements.

First, verify what executed. The wall-time ratio against a CPU baseline is a cheap screening check for silent CPU fallback; follow it with execution-placement checks in the server process, and treat `--list-devices` output as evidence of nothing.

Second, calibrate portability expectations. GPU-versus-local-CPU agreement was 18% at Q4, 35% at Q8, and 88% at F16, with cross-build and cross-class CPU comparisons showing the same ordering — while same-class CPU host pairs agreed completely. Do not assume exact-output portability when changing numerical implementations at Q4-class precision.

Third, pin the stack for reproducibility. The identical-stack controls reproduced every output they tested. If you need reproducible output, pin the backend, binary, GPU, and driver — and specify the complete execution configuration when you claim reproducibility.

Fourth, do not treat default-tool perplexity as evidence of behavioral equivalence.

I release the harness, the per-token logprob dataset, and the audit tooling at `https://github.com/shashb27/llamacpp-backend-conformance` (dataset: v1.0 release).

Acknowledgments

This study was enabled end-to-end by OxCapsule [13], OXMIQ Labs' fleet-management and remote-execution platform for heterogeneous AI compute (`https://oxmiq.ai/oxcapsule`): the 12-machine, three-vendor, two-OS fleet was swept, checkpointed, and audited from a single workstation. The author is a Product Lead at OXMIQ Labs, which develops OxCapsule; the platform is named here for transparency about how the measurements were produced.

Use of AI tools. Generative AI tools were used, under the author's direction and review, in preparing this manuscript and in developing the accompanying analysis code. The author is responsible for all content. The analysis pipeline is fully released and deterministic: every number reported here is recomputable from the public raw dataset, and the headline results were reproduced by two independently developed recomputation scripts.

References

[1] A. Schlögl, N. Hofer, and R. Böhme. Causes and effects of unanticipated numerical deviations in neural network inference frameworks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[2] H. He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Blog post, September 2025. `https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/`

[3] J. Yuan, H. Li, X. Ding, W. Xie, Y.-J. Li, W. Zhao, K. Wan, J. Shi, X. Hu, and Z. Liu. Understanding and mitigating numerical sources of nondeterminism in LLM inference. *arXiv preprint* arXiv:2506.09501 (v2), 2025.

[4] A. Dutta, S. Krishnan, N. Kwatra, and R. Ramjee. Accuracy is not all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2407.09141.

[5] D. Pape, J. Evertz, and L. Schönherr. The silent hyperparameter: Quantifying the impact of inference backends on LLM reproducibility. *arXiv preprint* arXiv:2605.19537, 2026.

[6] R. Gond, A. K. Kamath, R. Ramjee, and A. Panwar. LLM-42: Enabling determinism in LLM inference with verified speculation. *arXiv preprint* arXiv:2601.17768, 2026.

[7] G. Du, A. N. Khan, R. Zhou, X. Liu, D. Chakrabarti, F. Suya, and X. Li. Greedy decoding is not precision-invariant: Cross-precision output divergence in LLM inference. *Transactions on Machine Learning Research (TMLR)*, 2026. arXiv:2609.26621.

[8] L. Cooper, S. Jeong, H. Jeon, J. Young, and H. Kim. Accelerating the mitigation of LLM inference nondeterminism across GPU architectures. *arXiv preprint* arXiv:2609.25624, 2026.

[9] J. M. Ong, M. Di Ferrante, A. Pazdera, R. Garner, S. Jaghouar, M. Basra, M. Ryabinin, and J. Hagemann. TOPLOC: A locality sensitive hashing scheme for trustless verifiable inference. *arXiv preprint* arXiv:2501.16007, 2025.

[10] H. V. Pham, T. Lutellier, W. Qi, and L. Tan. CRADLE: Cross-backend validation to detect and localize bugs in deep learning libraries. In *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2019. doi:10.1109/ICSE.2019.00107.

[11] D. Zhuang, X. Zhang, S. L. Song, and S. Hooker. Randomness in neural network training: Characterizing the impact of tooling. *arXiv preprint* arXiv:2106.11872, 2021.

[12] H. V. Pham, S. Qian, J. Wang, T. Lutellier, J. Rosenthal, L. Tan, Y. Yu, and N. Nagappan. Problems and opportunities in training deep learning software systems: An analysis of variance. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 771–783, 2020. doi:10.1145/3324884.3416545.

[13] OXMIQ Labs. OxCapsule: Fleet management and inference for heterogeneous AI compute. `https://oxmiq.ai/oxcapsule`. Public beta, November 2025; accessed October 2026.